



BIT1123/BISE2093/DIT1113

OBJECT ORIENTED PROGRAMMING

ASSIGNMENT 1 - INDIVIDUAL

(20%)

ALL SECTIONS

FACULTY OF INFORMATION TECHNOLOGY

CITY UNIVERSITY MALAYSIA

CYBERJAYA CAMPUS

PREPARED BY

SIR NAZMIRUL IZZAD BIN NASSIR

Name	NURAIN BADRISYIA BINTI MOHD GHAZALI
Student ID	202307010013
Class Code	202605F0780
Program	BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) (HONS)
NRIC	050622030614

Table of Contents

Introduction.....	3
Knowledge gained from each tutorial	3
Challenges encountered.....	6
How the challenges were overcome	6
Improvements in Java programming skills	7
Understanding of Object-Oriented Programming concepts	8
Future learning plans	9
Conclusion	9
GitHub Repository URL	10

Introduction

This assignment reflects my learning journey throughout the BIT1123 Object-Oriented Programming course. During the tutorials, I learned Java programming step by step, starting from basic Java syntax and GitHub usage and gradually moving to Object-Oriented Programming concepts such as classes, objects, inheritance, encapsulation, abstraction, file handling, and graphical user interfaces.

The tutorials gave me opportunities to practise coding rather than only learning the concepts theoretically. Each week introduced a different programming concept, and I was able to apply the concepts through practical activities. This helped me understand how Java can be used to develop simple applications and solve programming problems.

Knowledge gained from each tutorial

- Week 1: Getting Started with Codespaces via GitHub

In Week 1, I learned how to create and manage a GitHub repository for my programming tutorials. I also learned how to use GitHub Codespaces as a development environment. Before this tutorial, I was more focused on writing code, but this activity showed me that organising and managing code is also an important part of programming.

I learned basic Git commands such as `git status`, `git add`, `git commit`, and `git push`. I also learned how to create folders and organise Java files inside a repository. This became useful for keeping my tutorial work organised throughout the semester.

- Week 2: Java Classes, Objects and Constructors

Week 2 introduced me to the basic structure of Object-Oriented Programming using Java. I worked with a `Student` class containing attributes such as `name`, `age` and `GPA`.

I learned how a constructor can be used to initialise the values of an object and how methods can be created to perform specific actions. For example, the `displayInfo()`, `study()`, and `takeExam()` methods helped me understand that an object can contain both data and behaviour. This tutorial helped me understand the basic relationship between a class and an object.

- **Week 3 & 4: Inheritance and Method Overriding**

Week 3 and Week 4 introduced inheritance. The tutorial used a university scenario involving a Person, Student and Lecturer. I learned that a child class can inherit properties and behaviour from a parent class instead of having to write the same code again.

I also learned about method overriding. The Student and Lecturer classes could provide their own version of the introduce() method. This helped me understand how different objects can behave differently even when they come from the same parent class. I also became more familiar with extends and super() when creating relationships between classes.

- **Week 5 – Encapsulation**

Week 5 focused on encapsulation through the Student Information System. I learned why variables such as student ID, name and CGPA should not simply be made public.

Using private variables together with getters and setters helped me understand how access to an object's data can be controlled. At first, getters and setters seemed unnecessary because the values could be accessed more directly, but after completing the activity, I understood that encapsulation provides better control over how data is accessed and modified.

- **Week 6: Employee and Lecturer**

In Week 6, I continued practising inheritance through an employee and lecturer scenario. I learned how a Lecturer can inherit common information such as ID and name from an Employee class while having additional information such as subject.

This tutorial helped me understand how inheritance can reduce repeated code. Instead of declaring the same employee information again in the lecturer class, the lecturer could reuse the properties and constructor from the parent class using super().

- **Week 7: Smart Home Appliance Management System**

Week 7 introduced abstraction through the Smart Home Appliance Management System. I learned how an abstract class can define common functionality for different appliances. For example, different appliances can all be turned on and turned off, but the way each appliance operates can be different. The Appliance class therefore provides common behaviour while individual appliance classes can provide their own implementation. This tutorial helped me understand the purpose of abstraction and why it is useful when designing programs with different objects that share common characteristics.

- **Week 8–9: To-Do List and File Handling**

Week 8 and Week 9 were interesting because the tutorial created a more practical application. I learned how to use an ArrayList to store multiple tasks entered by a user. I then extended the program by saving the tasks into a text file and reading the tasks back from the file. This introduced me to file handling using classes such as BufferedWriter and BufferedReader. This tutorial helped me understand that programs can work with information beyond temporary data stored while the program is running. The ability to save and reload tasks made the program feel more like a real application.

- **Week 10: Programming Quiz Battle GUI**

Week 10 was one of the most interesting tutorials because I worked with a graphical user interface using Java Swing. The Programming Quiz Battle application displayed a question and provided buttons for the possible answers.

I learned about components such as JFrame, JLabel and JButton, as well as event handling using ActionListener. The program could respond to the user's button selection and determine whether the selected answer was correct. This tutorial showed me that Java can be used to create interactive applications rather than only terminal-based programs.

Challenges encountered

One of the main challenges I experienced was understanding Java syntax while simultaneously trying to understand new programming concepts. Sometimes I knew what I wanted the program to do, but I was unsure about the correct Java syntax to implement it.

Furthermore, inheritance was also challenging at first, especially understanding the relationship between a parent class and a child class. I initially found `extends` and `super()` confusing because I had to understand which attributes and methods belonged to the parent class and which belonged to the child class. Encapsulation was another concept that required some adjustment. I initially wondered why variables needed to be declared as `private` when they could be accessed more easily if they were `public`. After working with getters and setters, I gradually understood the purpose of controlling access to data.

In addition, the Week 8&9 file-handling activity was also challenging because it involved several new concepts at the same time. I had to understand `ArrayList`, file writing, file reading, loops and exception handling. The Week 10 GUI activity was challenging in a different way because I had to understand how buttons and events interact with the program. It was not only about writing statements that run from top to bottom and the program also needed to respond to the user's actions.

How the challenges were overcome

I overcame these challenges mainly through repeated practice and checking my code carefully. Instead of trying to complete the entire program at once, I broke the problem into smaller parts and focused on understanding each section. For example, when working with inheritance, I first identified the common attributes and behaviour of the parent class before looking at what was unique to the child class. This made the relationship between `Person`, `Student` and `Lecturer` easier to understand. When I encountered syntax errors, I checked the structure of my classes, constructors, methods, brackets, and variable names. I also compared the program output with the expected output provided in the tutorials. This helped me identify where my implementation was different from what was expected.

For the file-handling activity, I approached the problem step by step. First, I made sure the tasks could be stored in the ArrayList. After that, I worked on saving the tasks to the file and finally loading them again. Breaking the activity into smaller sections made it easier to understand. For the GUI activity, I focused on understanding what each component was responsible for. I learned that the buttons were not only visual elements; they also needed an event listener to respond when the user clicked them. Overall, debugging became an important part of my learning process. Instead of seeing an error as simply a problem, I started using it as a way to understand what was wrong with my code.

Improvements in Java programming skills

My Java programming skills improved gradually throughout the tutorials. At the beginning, I was mainly learning basic syntax and how to write simple Java statements. After completing the tutorials, I became more comfortable creating classes, objects, constructors, attributes and methods.

I also improved my ability to structure programs into different classes instead of putting everything into one large program. This made me understand that good programming involves organising code into logical components. My debugging skills also improved. Previously, an error in the program could be confusing, but through the tutorials I became more comfortable reading error messages and checking my code to find the cause.

I also gained experience with more advanced Java features such as inheritance, encapsulation, abstraction, ArrayList, file handling, and Swing GUI development. Another improvement was my ability to use GitHub. I learned how to organise my projects into folders and use Git commands to save and update my programming work. This gave me experience beyond just writing Java code and helped me understand the importance of maintaining a proper programming project.

Understanding of Object-Oriented Programming concepts

The tutorials helped me understand the main concepts of Object-Oriented Programming more clearly through practical examples.

- **Classes and Objects**

I learned that a class acts as a blueprint for creating objects, while an object is an instance of a class. The Student example helped me understand this because the class defines information such as name, age, and GPA, while an actual student object contains specific values.

- **Encapsulation**

Week 5 helped me understand encapsulation. By making variables private and accessing them through getters and setters, the internal data of an object can be protected and controlled.

- **Inheritance**

Week 3–4 and Week 6 helped me understand inheritance. A child class can reuse properties and methods from a parent class. For example, a Student can inherit from Person, while a Lecturer can inherit from Employee.

- **Polymorphism**

The Week 3–4 activity also helped me understand the idea of polymorphism, particularly through method overriding. Although different classes inherit from the same parent, they can provide different implementations of the same method.

- **Abstraction**

Week 7 introduced abstraction. I learned that common functionality can be defined while the specific implementation is left to individual classes. The appliance example made this easier to understand because different appliances can operate differently while still sharing common actions such as turning on and turning off.

Future learning plans

After completing these tutorials, I would like to continue improving my Java programming skills by building larger applications instead of only completing small tutorial exercises. One area I would like to improve is Java GUI development. Week 10 introduced me to Swing, and I would like to learn how to create more complete interfaces with better layouts, multiple screens, input validation and more interactive features.

I also plan to improve my understanding of file handling and databases so that I can create applications that store and manage larger amounts of information. Another area I want to develop is debugging and problem-solving. I want to become better at identifying the cause of errors independently instead of relying heavily on examples.

In the future, I would also like to learn how to combine multiple OOP concepts in one complete Java application. For example, I could create a system that uses inheritance, encapsulation, abstraction, collections, file handling, and a GUI together.

Conclusion

Overall, the BIT1123 tutorials provided me with a gradual introduction to Java and Object-Oriented Programming. I started with basic Java programming and GitHub usage and progressed towards more advanced concepts such as inheritance, encapsulation, abstraction, file handling and GUI programming.

The tutorials were sometimes challenging, especially when several new concepts were introduced together. However, working through the activities and debugging my code helped me improve both my programming skills and my confidence. The most important lesson I gained is that learning programming requires practice, patience, and problem-solving. Understanding OOP concepts became easier when I could see them being applied to actual programming scenarios.

By completing these tutorials, I have developed a stronger foundation in Java and a better understanding of how Object-Oriented Programming can be used to organise and develop software. I believe the knowledge and skills gained from this assignment will be useful for future programming subjects and larger software development projects.

GitHub Repository URL

https://github.com/nainghazali-Kshahrewar-lab/NURAIN-BADRISYIA_object-oriented-programming-tutorials.git